

Go Log File Analyzer

Complete Assignment Specification

1. Assignment Overview

Build a command-line tool that reads, parses, filters, and analyzes log files. The tool should support multiple output formats, filtering by log level and timestamp, and provide statistics about the logs. This assignment will reinforce your understanding of file I/O, interfaces, error handling, goroutines, and command-line flag parsing.

2. Core Requirements

2.1 Log Entry Structure

Each log entry follows this format: [TIMESTAMP] [LEVEL] MESSAGE

Example: [2024-01-15 10:30:45] ERROR Database connection failed

- **TIMESTAMP:** ISO format (YYYY-MM-DD HH:MM:SS)
- **LEVEL:** One of INFO, WARNING, ERROR (case-insensitive)
- **MESSAGE:** Anything after the level (can contain spaces, special chars)

2.2 Data Structures

Define these types in your code:

```
type LogLevel string // INFO, WARNING, ERROR

const (
    LevelInfo LogLevel = "INFO"
    LevelWarning LogLevel = "WARNING"
    LevelError LogLevel = "ERROR"
)

type LogEntry struct {
    Timestamp time.Time // Parse as: 2006-01-02 15:04:05
    Level LogLevel
    Message string
}

type LogFile struct {
    Entries []LogEntry
    Path string
}
```

2.3 Parsing Requirements

Input Format: Read log file line by line. Each line is ONE log entry.

Parsing Rules:

- Extract timestamp from [YYYY-MM-DD HH:MM:SS] format
- Extract level from [LEVEL] (case-insensitive, normalize to uppercase)
- Everything after the level is the message
- If a line doesn't match format, skip it or log a warning
- Use Go's `time.Parse()` with layout `"2006-01-02 15:04:05"`

Example parsing:

Input: `[2024-01-15 10:30:45] ERROR Database failed`

- Timestamp: 2024-01-15 10:30:45
- Level: ERROR
- Message: "Database failed"

2.4 Formatter Interface

Define a `Formatter` interface that different output formats implement:

```
type Formatter interface {  
    Format(entry LogEntry) string  
}
```

Implementations required:

TextFormatter: Simple text output

Output: `[2024-01-15 10:30:45] ERROR Database failed`

JSONFormatter: JSON format

Output:

```
{"timestamp": "2024-01-15T10:30:45Z", "level": "ERROR", "message": "Database failed"}
```

CSVFormatter (Bonus): Comma-separated

Output: `2024-01-15 10:30:45,ERROR,Database failed`

2.5 Filtering Requirements

Implement a `Filter` function that takes:

```
func (lf *LogFile) Filter(level LogLevel, startTime, endTime time.Time)  
[]LogEntry
```

Rules:

- If level is empty string, include all levels

- If startTime is zero value, no lower bound
- If endTime is zero value, no upper bound
- Filters are AND-ed together (must match all criteria)
- Return a new slice with matching entries

2.6 Statistics Requirements

Implement a Statistics method:

```
type Stats struct {
    TotalEntries int
    InfoCount int
    WarningCount int
    ErrorCount int
    MostCommonError string // The error message that appears most
}

func (lf *LogFile) GetStatistics() Stats
```

Details:

- Count total and by level
- For MostCommonError, find ERROR level entries and return the message that appears most frequently
- If no errors, set to empty string

3. Command-Line Flags

Use Go's `flag` package. Support these flags:

Required:

- `-file` string: Path to log file (required)

Optional:

- `-level` string: Filter by level (INFO, WARNING, ERROR, or empty for all)
- `-from` string: Start timestamp (format: YYYY-MM-DD HH:MM:SS)
- `-to` string: End timestamp (format: YYYY-MM-DD HH:MM:SS)
- `-format` string: Output format (text, json, csv). Default: text
- `-stats`: Print statistics instead of entries

Usage Examples:

```
go run main.go -file=app.log
go run main.go -file=app.log -level=ERROR
go run main.go -file=app.log -level=ERROR -format=json
go run main.go -file=app.log -from="2024-01-15 10:30:00" -to="2024-01-15 11:00:00"
go run main.go -file=app.log -stats
```

4. Step-by-Step Implementation

Step 1: Setup

- Create `main.go`
- Define all types (`LogLevel`, `LogEntry`, `LogFile`, `Stats`, `Formatter` interfaces)
- Create `app.log` test file with sample entries

Step 2: File Reading

- Implement a function to read file and return `[]string` (one per line)
- Handle file open/close errors

Step 3: Parsing

- Implement `ParseLogEntry(line string) (*LogEntry, error)`
- Parse timestamp, level, message
- Handle invalid formats gracefully

Step 4: LogFile struct

- Implement `NewLogFile(path string) (*LogFile, error)`
- Reads file, parses all entries, returns populated `LogFile`

Step 5: Formatters

- Implement `Format()` method for `TextFormatter`
- Implement `Format()` method for `JSONFormatter`

Step 6: Filtering

- Implement `Filter()` method
- Test with different combinations

Step 7: Statistics

- Implement GetStatistics() method

Step 8: Main function

- Parse flags
- Load log file
- Apply filters if requested
- Format and print output

5. Sample Test Data

Create app.log with this content:

```
[2024-01-15 10:30:45] INFO Application started
[2024-01-15 10:31:02] INFO User login: admin
[2024-01-15 10:32:15] WARNING High memory usage detected
[2024-01-15 10:33:00] ERROR Database connection failed
[2024-01-15 10:33:05] ERROR Retrying connection...
[2024-01-15 10:33:10] INFO Connection restored
[2024-01-15 10:35:22] WARNING Cache miss rate 45%
[2024-01-15 10:36:00] ERROR Invalid request format
[2024-01-15 10:36:15] INFO Request processed
[2024-01-15 10:37:00] ERROR Timeout on external API call
```

6. Expected Outputs

Command: go run main.go -file=app.log -stats

Output: Total: 10 | INFO: 4 | WARNING: 2 | ERROR: 4 | Most Common Error: Database connection failed

Command: go run main.go -file=app.log -level=ERROR -format=json

Output (one per line):

```
{"timestamp": "2024-01-15T10:33:00Z", "level": "ERROR", "message": "Database connection failed"}
{"timestamp": "2024-01-15T10:33:05Z", "level": "ERROR", "message": "Retrying connection..."}
{"timestamp": "2024-01-15T10:36:00Z", "level": "ERROR", "message": "Invalid request format"}
{"timestamp": "2024-01-15T10:37:00Z", "level": "ERROR", "message": "Timeout on external API call"}
```

Command: go run main.go -file=app.log -from="2024-01-15 10:33:00" -to="2024-01-15 10:36:00"

Output: All entries between those timestamps (inclusive)

7. Error Handling

File not found:

Print: "Error: file not found: [path]"

Invalid log line:

Skip the line and continue (or optionally log a warning)

Invalid timestamp in filter:

Print error and exit

Invalid flags:

Go's flag package handles this automatically

8. Bonus Challenges

Difficulty: Easy

- Add -keyword flag to search message content
- Add -top N flag to show top N most common errors

Difficulty: Medium

- Support reading from multiple files concurrently (use goroutines)
- Add RegEx support for message filtering: -regex pattern
- Implement CSVFormatter

Difficulty: Hard

- Create a second LogFilter interface and different filter implementations
- Add real-time file monitoring (watch for new entries)
- Implement a Reader interface wrapper that decompresses gzip logs

9. Testing Checklist

Parse Tests:

- Valid log line parses correctly
- Invalid lines are skipped
- All log levels are recognized (case-insensitive)

Filter Tests:

- Filter by level only
- Filter by timestamp range
- Filter by both level and timestamp
- Empty filters return all entries

Format Tests:

- TextFormatter outputs correct format
- JSONFormatter outputs valid JSON
- Timestamps in JSON are ISO 8601 format

Statistics Tests:

- Counts are accurate
- Most common error is correct

CLI Tests:

- All flags work individually
- Flags work in combination
- Errors are handled gracefully

10. Helpful Go Features & Packages

Packages you'll need:

```
import (  
    "flag"           // Command-line flags  
    "fmt"           // Printing  
    "os"            // File I/O  
    "bufio"         // Line-by-line reading  
    "strings"       // String manipulation  
    "time"          // Timestamp parsing  
    "encoding/json" // JSON marshaling  
)
```

Key functions:

- `bufio.Scanner` for reading lines
- `strings.Fields()` or `strings.Index()` for parsing
- `time.Parse(layout, value string)` for timestamps
- `json.Marshal()` for JSON output
- `flag.String()`, `flag.Bool()`, etc. for CLI

Time layout string:

Go uses reference time: Mon Jan 2 15:04:05 MST 2006

For "2024-01-15 10:30:45" use layout: "2006-01-02 15:04:05"

11. Implementation Tips

- Start simple: get file reading and basic parsing working first
- Test each function independently before integrating
- Use interfaces early—they make your code flexible
- Handle edge cases: empty files, malformed lines, etc.
- When parsing timestamps, always check the error from `time.Parse()`
- For JSON, remember to use `json.Marshal()` which returns `[]byte`
- Use maps for statistics (count occurrences by mapping string → int)
- Test filters by creating cases that should include/exclude entries